

Supplementary Information

Self-driving microscopy benchmarks support qualification but not task-independent configuration selection for large language model agents

Nathan S. Johnson¹ and Ian Abshire¹

¹Carl Zeiss Research Microscopy Solutions, 5300 Central Boulevard, Dublin, CA 94568

1 Agent state model

All nodes in the graph operated on a shared mutable **AgentState** dictionary. This state provided the working memory of the system while limiting dependence on an indefinitely growing conversation history. Table 1 summarizes the principal fields.

2 MCP server implementation

The controller used six MCP servers. Each server was implemented as a separate Python service with a bounded responsibility. This separation allowed the software requirements and safety checks for instrument control, image processing, and free-form code execution to be managed independently.

2.1 Versa instrument-control server

The **Versa** server wrapped the ZEISS vendor API (`XradiaPy.Core.XRM`) and exposed deterministic tools for instrument-state queries, X-ray source control, axis motion, filter and objective selection, and image acquisition. Each acquisition wrote an image and a JSON metadata sidecar containing the instrument state. The sidecar was used by downstream geometry and image-analysis tools.

The server exposed small, composable actions rather than a monolithic workflow command. Higher-level sequencing remained in the LangGraph controller, where planning, retries, and agent coordination could be recorded in the workflow trace.

2.2 Recipe server

The **Versa-Recipe** server wrapped the ZEISS recipe API (`XradiaPy.Recipe.Recipe`). It modified acquisition parameters, wrote updated recipe files to the run workspace, opened and executed recipes, and emitted JSON snapshots of the recipe state. Recorded fields included the objective, filter, angular range, number of projections, and acquisition parameters.

2.3 Geometry and optics server

The **Geometry** server performed calibration-aware calculations for sample centering and imaging geometry. Centering tools converted image-space centroids into stage-space corrections using the image shape, pixel size, stage position, sample rotation, and fixed sign conventions stored in the tool implementation. This removed the need for the LLM to remember axis mappings or calculate movement signs from free-form context.

The server also selected source and detector positions to satisfy a requested field of view or pixel size while respecting calibrated mechanical and detector limits. It returned either a feasible configuration or a structured report explaining why no solution was available.

State key	Type	Description
<code>messages</code>	<code>list[Message]</code>	Bounded transcript containing human, AI, and tool messages. Older messages may be pruned or summarized to enforce token and message-count budgets.
<code>next</code>	<code>str</code>	Next uncompleted task in the current workflow.
<code>run_id</code>	<code>str</code>	Unique identifier assigned at the beginning of the workflow.
<code>working_directory</code>	<code>str</code>	Active directory used for file operations.
<code>tasks</code>	<code>List[dict]</code>	Structured task records containing identifiers, descriptions, assignments, status, and optional results.
<code>assigned_tasks_for_agent</code>	<code>List[str]</code>	Tasks delegated by the supervisor to a worker agent.
<code>agent_completed_tasks</code>	<code>List[str]</code>	Tasks reported as complete by a worker agent.
<code>retry_count</code>	<code>int</code>	Number of attempts made on the active task.
<code>last_error</code>	<code>str</code>	Most recent tool or workflow error.
<code>results</code>	<code>List[dict]</code>	Structured results returned by MCP tool calls.
<code>files</code>	<code>List[str]</code>	Files created during the workflow, including images, volumes, JSON metadata, recipes, and plots.
<code>shared_data</code>	<code>Dict[str, object]</code>	Structured values passed between agents, including file keys, instrument positions, and analysis results.
<code>prior_summary</code>	<code>List[str]</code>	Compressed summaries of messages removed from the active transcript.

Table 1: Representative **AgentState** schema used by the LangGraph controller.

2.4 Image-analysis server

The **ImageAnalysis** server exposed deterministic image-processing operations used for centering, transmission measurement, and workflow verification. The server used **numpy**, **Pillow**, **scikit-image**, and **matplotlib**. Tools included TIFF loading, Otsu thresholding, connected-component measurements, centroids, bounding boxes, histograms, and annotated image generation.

2.5 File-handling server

The **FileReading** server provided guarded file operations, including directory listing, file-existence checks, JSON and text reading, and controlled path resolution. Relative paths were resolved from the active working directory, and errors were returned as structured messages rather than raw tracebacks.

2.6 Python-execution server

The **python_executor** server accepted a Python source string, optionally saved the code in the run artifacts directory, and executed it in a subprocess while recording standard output, standard error, and wall-clock time. Dedicated MCP tools were preferred for repeated scientific operations. Free-form execution was reserved for analyses or plots that were not already represented by validated tools.

3 Agent prompts

The one-agent graph used one prompt for a ReAct agent with access to all six MCP servers. The two-agent graph used a supervisor and one unified worker. The three-agent graph used a supervisor with separate instrument and image-analysis workers. Table 2 summarizes the roles. The exact Python source files supplied for this study are included with the manuscript package. The full prompt listings are included in the code repository.

Topology	Prompt role	Responsibility encoded in the prompt
One-agent	Unified ReAct agent	Plans the complete workflow, controls the instrument, handles files and recipes, performs image analysis, and executes Python.
Two-agent	Supervisor	Creates and updates the task list, delegates related batches to the unified worker, and determines when the workflow is complete.
Two-agent	Unified worker	Executes instrument-control, recipe, file, geometry, image-analysis, and Python tasks assigned by the supervisor.
Three-agent	Supervisor	Creates the task plan and assigns consecutive instrument or image-analysis tasks to the corresponding specialist.
Three-agent	Instrument worker	Executes microscope motion, X-ray control, image acquisition, recipe creation, and recipe execution.
Three-agent	Image-analysis worker	Processes TIFF images, performs segmentation and measurement, writes analysis files, and executes custom Python.

Table 2: System-prompt roles used by the three agent graph topologies.

Outcome or metric	Definition
Required tool	The workflow must call a specified MCP tool.
Required text output	The final response must contain a specified value or string.
Required agent	A specified agent must be visited in graph topologies where responsibility is divided among agents.
File creation	A file with the required name or extension must be created in the run workspace.
File contents	A created text or JSON file must contain a specified value or field.
Quantitative result	A measured or calculated value must fall within the tolerance defined for the test.
Forbidden tool	A specified tool must not be called during the workflow.
Duration	Wall-clock time required to complete the test.
Input and output tokens	Total LLM token use recorded during the workflow.
Estimated cost	Estimated API cost calculated from recorded token use and the model-specific pricing configured in the benchmark framework.
LLM calls	Number of model invocations during the workflow.
Tool calls	Number of MCP tool invocations during the workflow.
Retrieval identifiers	Stable identifier for every RAG entry retrieved during the workflow.

Table 3: Pass/fail checks and operational metrics recorded by the benchmark framework. Tests used different combinations of checks depending on their intended purpose.

4 Benchmark definitions

The benchmark used a composite pass definition. A run passed only when all checks defined for that test were satisfied. The available checks and operational metrics recorded for every run are summarized in Table 3.

4.1 Benchmark suites

The 53 tests were divided into seven source YAML files. Some suites were intended primarily for qualification and regression, while others combined several operations into integrated workflows. Table 4 summarizes the suite structure. The complete YAML definitions are included, in full, in the code repository.

4.2 Golden end-to-end suite

The six-test `tdp_workflow` suite was used as a repeated end-to-end benchmark. Table 5 is a compact summary; the YAML listings in manuscript repository are the authoritative definition.

YAML suite	Role	No. tests	Task scope	Primary pass/fail basis
tdp_workflow.yaml	Golden suite	6	End-to-end 3D-printed artifact workflow covering setup, centering, sample sizing, geometry selection, recipe creation and execution, and safe shutdown.	Required actions, artifact creation and verification, forbidden-tool checks, and operational limits.
integration_workflows.yaml	Supporting	12	Multi-tool workflows combining setup, acquisition, image analysis, survey imaging, transmission or exposure calculation, and complete imaging sequences.	Required actions and artifacts together with LLM-call, tool-call, duration, and timeout limits.
unit_instrument.yaml	Supporting	19	Atomic control tests for source state, stage motion, objective and filter selection, and image acquisition.	Required instrument action and operational limits.
geometry_recipe.yaml	Supporting	6	Geometry and recipe tests for magnification, centering transforms, recipe modification, and recipe execution.	Required calculations, tool calls, and artifact verification.
regression_performance.yaml	Supporting	8	Regression, idempotency, safety, timing, and stress tests for known failure modes and performance bounds.	Required action and operational limits.
unit_image.yaml	Supporting	1	Image-analysis test using custom Python after acquisition.	Required acquisition and analysis behavior.
debug.yaml	Supporting	1	Minimal smoke test for benchmark execution.	Successful tool execution.

Table 4: Machine-readable benchmark suites used to evaluate the agentic microscope controller.

5 Study-specific efficiency score

A study-specific efficiency score was used during configuration screening to combine pass rate with resource use. The score included mean duration, mean token use, estimated cost, LLM calls, tool calls, and the number of failures. Table 6 lists the relative weights used in the analysis. This score was used as a practical utility function for the present system and was not intended as a general definition of agent efficiency.

6 Physical validation definitions

The five holdout validation tasks were defined separately from the 53-test benchmark. Table 7 summarizes the expected outputs and numerical criteria. A normalized machine-readable definition is included in manuscript repository.

Test ID	Test name	Purpose	Principal pass criteria	Limits
tdp_001	setup_instrument_tdp	Set up the instrument for general imaging and acquire an image without plotting.	Call <code>turn_xray_on</code> , <code>move_axis_absolute</code> , and <code>single_acquisition</code> ; at least five tool calls; do not call <code>execute_python_code</code> or <code>plot_tiff</code> .	Duration ≤ 600 s; LLM calls ≤ 20 ; tool calls ≤ 15 ; timeout 600 s.
tdp_002	center_sample_tdp	Center the sample in the field of view and save the final stage position.	Call <code>movement</code> , <code>acquisition</code> , <code>Otsu segmentation</code> , and <code>XY/YZ recentring tools</code> ; create <code>centered_position.json</code> .	Duration ≤ 300 s; LLM calls ≤ 30 ; tool calls ≤ 20 ; timeout 800 s.
tdp_003	estimate_size_tdp	Estimate sample dimensions from two orthogonal views.	Acquire and segment images; at least seven tool calls; create <code>sample_size.json</code> .	Duration ≤ 300 s; LLM calls ≤ 30 ; tool calls ≤ 20 ; timeout 1000 s.
tdp_004	calculate_magnification_tdp	Calculate source and detector positions for a requested 100- μm pixel size.	Read <code>sample_size.json</code> ; call <code>geometry</code> and <code>Python-execution tools</code> ; create <code>magnification_settings.json</code> .	Duration ≤ 300 s; LLM calls ≤ 20 ; tool calls ≤ 10 ; timeout 600 s.
tdp_005	create_run_recipe_tdp	Create and execute a recipe using the centered position and magnification settings.	Call <code>recipe modification</code> and <code>execution tools</code> ; at least three tool calls; create <code>modified_recipe.rcp</code> .	Duration ≤ 600 s; LLM calls ≤ 20 ; tool calls ≤ 9 ; no separate timeout is specified in the YAML.
tdp_end	end_testing_tdp	Turn off X-rays and return the stage, source, and detector to specified positions.	Call <code>movement</code> and <code>turn_xray_off tools</code> ; at least four tool calls.	Duration ≤ 180 s; LLM calls ≤ 20 ; tool calls ≤ 15 ; timeout 600 s.

Table 5: Compact summary of the golden end-to-end benchmark suite.

Metric	Pass rate	Mean duration	Mean tokens	LLM calls	Tool calls	No. failures
Weight	0.4	0.15	0.15	0.1	0.1	0.1

Table 6: Weights used in the study-specific efficiency score.

Test ID	Required file	Required tools or workflow action	File-content or quantitative criterion
validation_001	setup_position.json	Turn on X-rays; acquire images; recent from sidcar; execute Python; at least eight tool calls.	Stage X $650 \pm 100 \mu\text{m}$; stage Y $11100 \pm 500 \mu\text{m}$; stage Z $0 \pm 100 \mu\text{m}$; source Z $-87 \pm 10 \text{ mm}$; detector Z $50 \pm 10 \text{ mm}$.
validation_002	sample_size.json	Acquire orthogonal views and execute Python; at least three tool calls.	<code>length_x</code> = $29 \pm 1 \text{ mm}$; <code>length_y</code> = $29 \pm 2 \text{ mm}$; <code>length_z</code> = $29 \pm 1 \text{ mm}$.
validation_003	optimal_exposure.json	Acquire an image and execute Python to select an exposure for approximately 5000 counts in the sample region.	<code>optimal_exposure</code> = $5.0 \pm 0.25 \text{ s}$.
validation_004	TDP_006_optimal.rcp	Read current positions and the saved exposure; call recipe modification and execution tools; begin acquisition.	Required recipe file exists.
validation_005	number_of_defects.json	Segment the latest reconstructed TIFF stack using multi-Otsu; plot slices through pores larger than 1 mm; execute Python.	<code>number_of_defects</code> equals the integer 5.

Table 7: Definitions of the five holdout physical validation tasks.

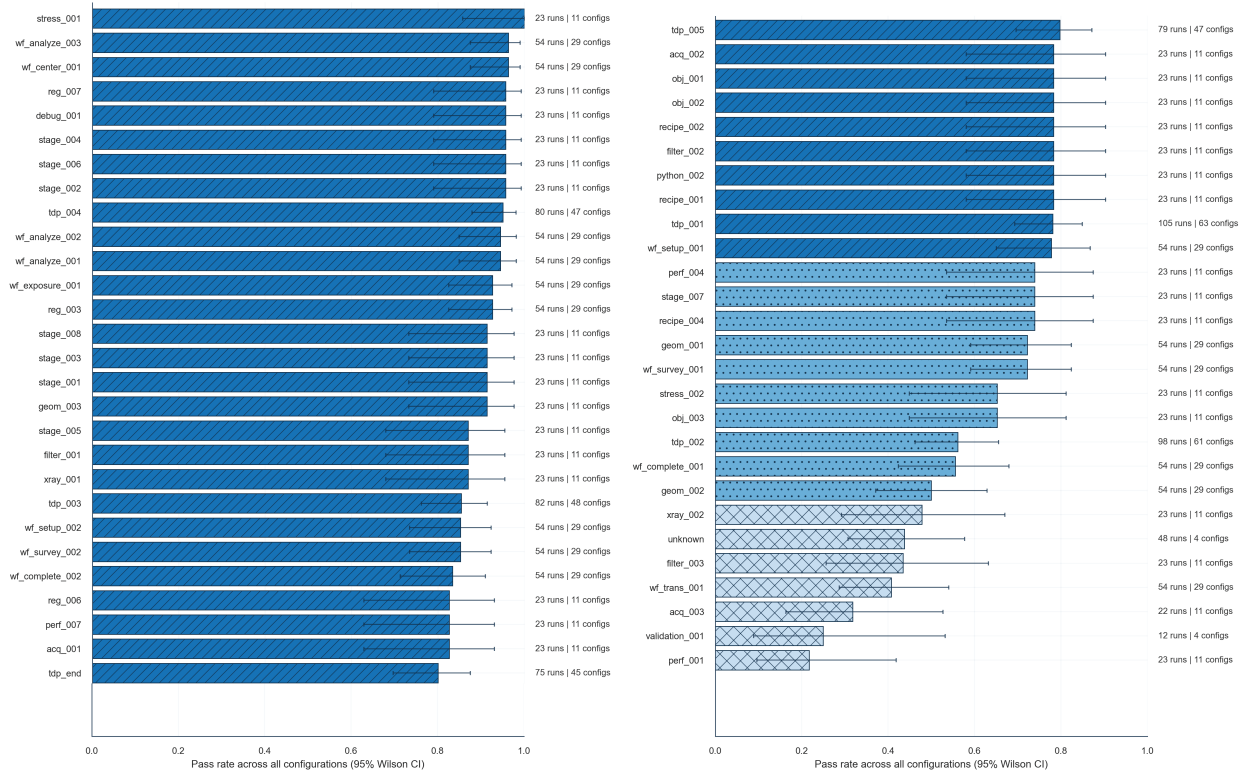


Figure 1: Composite pass rate for each of the 53 benchmark tests across the configurations on which it was evaluated. Error bars show 95% Wilson confidence intervals.